COMP101
Introduction to Programming
2025-26

Lecture-03
Systems lifecycle and ten stages of programming
Reading only

Systems Development Life Cycle (SDLC)

A. Strategic Study
B. Business Study
C. Feasibility Study
D. Requirements Analysis
E. Requirements Specification
F. Logical System Specification
G. Physical Design
H. Coding
I. Testing
J. Implementation
K. Maintenance

Creating a large computer system usually follows a life cycle:

A to F: Define the 'problem statement':
COMP101: this is the assignment as a 'problem specification' for which you program a solution

G, H, I: Program ('code') the specification in Python.
Done as 'ten stages of programming'
COMP101: addresses six of these stages

J, K: COMP101: Not applicable

G,H,I have ‘Ten Stages of Programming’ We use 3,5,6,7,8,9

1. Feasibility Study
   - Determine if the problem statement (‘requirement’) is suitable and achievable on computer.
   - May be quicker to do with pencil and paper with no expensive computer or programmer costs
   - In COMP101 our problem statements will be appropriate for solution by computer

2. Problem Specification (’Requirements Specification’ - a COMP101 assignment)
   - Solve the right problem as requested by the user

3. Program Design and Decomposition
   - Pseudocode the problem and identify alternatives for tackling the problem
   - Produce independent sub-tasks (functions etc) for manageability

4. Data Structure Specification and Implementation
   - Should be language-independent

The Ten Stages of Programming: We need (3,5,6,7,8,9)

5. Algorithm Selection and Analysis
   - Identify appropriate algorithm(s) to solve the problem
6. Code (syntactically and semantically correct)
   - Produce clear, readable code
7. Debug
   - Make a good attempt to make the program work
8. Testing and Formal Verification
   - No formal verification needed, but produce suitable test tables
9. Documentation
   - Provide appropriate documentation (or self documenting code)
10. Maintenance

Programming Languages

Python

Java

C#

C++

PROGRAMMING LANGUAGES

Machine code

Assembly

C

High level languages (many of them)
3rd generation
Abstractions of low level languages
Many different languages depending on what to be used for

Low level languages (few of them)
1st and 2nd generation
Direct access to the hardware
Handle memory management

Programming Languages

- 1st Generation (Low level)
  - Machine code (binary)
  - 01001000 01100101 01101100 01101100 01101111 00100001 (‘Hello’)
  - Has an ‘opcode’ (an instruction) and an ‘operand’ (data value to be used)
  - Does not need translating – already in the language of the hardware
- 2nd Generation (Low level – but ‘easier’ than 1st generation)
  - Assembly language uses mnemonics for opcodes and operands
  - LDA, STA, ADD, SUB, MOV etc
  - Needs to be translated into binary before execution – use an ‘assembler’

Programming Languages

- 3rd Generation (High level)
  - C
    - Often referred to as low level, mid level and high level language
    - has no automatic memory management
    - Manipulates hardware at a low level
    - Compiled language – an .exe file can execute directly on the CPU
  - C++
    - Extention of C – still low level manipulation but with object-oriented paradigm
    - Good performance for designing operating systems etc – especially gaming
    - Compiled language
  - C#
    - Extention again – object-oriented, more general purpose
    - Compiled language

Programming Languages

- 3rd Generation (High level)
  - Java
    - Good for web programming, cloud applications etc
    - Works across different platforms and devices
    - Compiled language – works faster
  - Python
    - Easy to learn and use
    - Works across different platforms and devices
    - Uses less code than Java
    - Interpreted language

Programming Languages Summary

- Low level
  - Closer to the language of the machine
  - Machine code does not need translating
  - Assembly needs an assembler to translate into machine code
  - C needs compiling into machine code
- High level
  - All need translating into machine code in order to execute
  - Translation is via a compiler or an interpreter
  - Interpreter reads code one line at a time and translates it
    - Code translated line-by-line at run-time
    - Errors are reported after each line – will show errors one at a time
    - Interpreted code fast for error analysis but slower execution than compiled code
  - Compiler reads all the code and then translates it
    - Code in its entirety compiled before run-time
    - Errors are reported at the end – will show every error all at once
    - Compiled code slower for error analysis but executes faster than interpreted code

Programming Languages Summary

- Source code (‘code’)
  - Program statements (high level or low level) saved in a file
  - Translated (interpreter, compiler, assembler) to machine code
- Object code
  - The output of source code after translation
  - Readable by the computer
- Linked code
  - Linker links library files etc prior to execution

https://www.bbc.co.uk>guides>zmthsrd>revision

Search: ‘BBC Bitesize program construction’

Programming Languages Summary

- Virtual Machines (JVM, PVM)
  - Software hosted on a computer to execute programs written on a different computer
  - Source code translated to object code
  - Object code translated to ‘bytecode’ (portable code - platform independent)
  - Pass the bytecode to the virtual machine for execution
  - Bytecode can be executed via an interpreter and can be further compiled to improve performance